

**Московский государственный технический
университет им. Н. Э. Баумана**

Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»
Отчет по ЛР №3-4. Python

Выполнила:
Студент группы ИУ5-31Б
Сербенюк Полина
Проверил:
Гапанюк Ю. Е.

2025 г.

`__init__.py`
"""

Пакет lab_python_fp - лабораторная работа по функциональному программированию
"""

```
from .field import field
from .gen_random import gen_random
from .unique import Unique
from .print_result import print_result
from .cm_timer import cm_timer_1, cm_timer_2
```

```
__all__ = [
    'field',
    'gen_random',
    'Unique',
    'print_result',
    'cm_timer_1',
    'cm_timer_2'
]
```

`Cm_timer.py`

```
import time
from contextlib import contextmanager
import sys
```

```
class cm_timer_1:
    def __enter__(self):
        self.start_time = time.time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        elapsed_time = time.time() - self.start_time
        print(f'time: {elapsed_time:.1f}')
```

```
@contextmanager
def cm_timer_2():
    start_time = time.time()
    try:
        yield
    finally:
```

```
elapsed_time = time.time() - start_time
print(f'time: {elapsed_time:.1f}')
```

```
if __name__ == "__main__":
    # Тесты
    print("Test cm_timer_1:")
    with cm_timer_1():
        time.sleep(0.5)

    print("\nTest cm_timer_2:")
    with cm_timer_2():
        time.sleep(0.5)
```

field.py

```
def field(items, *args):
    assert len(args) > 0

    if len(args) == 1:
        # Если передан один аргумент, возвращаем только значения
        key = args[0]
        for item in items:
            if key in item and item[key] is not None:
                yield item[key]
    else:
        # Если передано несколько аргументов, возвращаем словари
        for item in items:
            result = {}
            has_values = False
            for key in args:
                if key in item and item[key] is not None:
                    result[key] = item[key]
                    has_values = True
            if has_values:
                yield result

if __name__ == "__main__":
    # Тесты
    goods = [
        {'title': 'Ковер', 'price': 2000, 'color': 'green'},
        {'title': 'Диван для отдыха', 'color': 'black'},
```

```

        {'title': None, 'price': 3000},
        {'price': 2500, 'color': 'white'},
        {'title': 'Стул', 'price': None, 'color': 'brown'}
    ]

```

```

print("Test 1 - один аргумент:")
for item in field(goods, 'title'):
    print(item)

```

```

print("\nTest 2 - несколько аргументов:")
for item in field(goods, 'title', 'price'):
    print(item)

```

gen_random.py

```

import random

```

```

def gen_random(num_count, begin, end):
    for _ in range(num_count):
        yield random.randint(begin, end)

```

```

if __name__ == "__main__":
    # Тесты
    print("Test gen_random:")
    for num in gen_random(5, 1, 3):
        print(num, end=' ')
    print()

```

print_result.py

```

def print_result(func):
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        print(func.__name__)

        if isinstance(result, list):
            for item in result:
                print(item)
        elif isinstance(result, dict):
            for key, value in result.items():
                print(f'{key} = {value}')
        else:
            print(result)

```

```
    return result
```

```
    return wrapper
```

```
@print_result  
def test_1():  
    return 1
```

```
@print_result  
def test_2():  
    return 'iu5'
```

```
@print_result  
def test_3():  
    return {'a': 1, 'b': 2}
```

```
@print_result  
def test_4():  
    return [1, 2]
```

```
if __name__ == '__main__':  
    print('!!!!!!!')  
    test_1()  
    test_2()  
    test_3()  
    test_4()
```

```
process_data.py  
import json  
import sys  
from .field import field  
from .gen_random import gen_random  
from .unique import Unique  
from .print_result import print_result  
from .cm_timer import cm_timer_1  
  
path = sys.argv[1] if len(sys.argv) > 1 else 'data_light.json'
```

```
with open(path, encoding='utf-8') as f:
    data = json.load(f)
```

```
@print_result
def f1(arg):
    # arg - список словарей с вакансиями
    jobs = [item['job-name'] for item in arg if 'job-name' in item]
    # Убираем дубликаты (игнорируя регистр) и сортируем
    return sorted(set(job.lower() for job in jobs), key=str.lower)
```

```
@print_result
def f2(arg):
    # arg - список строк (профессий) из f1
    # Фильтруем только те, что начинаются с "программист"
    return [job for job in arg if job.lower().startswith('программист')]
```

```
@print_result
def f3(arg):
    # arg - отфильтрованный список профессий из f2
    # Добавляем "с опытом Python"
    return [f'{job} с опытом Python' for job in arg]
```

```
@print_result
def f4(arg):
    # arg - список профессий с Python из f3
    # Генерируем зарплаты и объединяем
    salaries = gen_random(len(arg), 100000, 200000)
    result = []
    for job, salary in zip(arg, salaries):
        result.append(f'{job}, зарплата {salary} руб.')
    return result
```

```
if __name__ == '__main__':
    with cm_timer_1():
        f4(f3(f2(f1(data))))
```

sort.py

```
def sort_by_abs(item):  
    return abs(item)
```

```
data = [4, -30, 100, -100, 123, 1, 0, -1, -4]
```

```
if __name__ == '__main__':  
    # Без lambda  
    result = sorted(data, key=sort_by_abs, reverse=True)  
    print(result)  
  
    # C lambda  
    result_with_lambda = sorted(data, key=lambda x: abs(x), reverse=True)  
    print(result_with_lambda)
```

unique.py

```
class Unique(object):  
    def __init__(self, items, **kwargs):  
        self.items = iter(items)  
        self.seen = set()  
        self.ignore_case = kwargs.get('ignore_case', False)  
  
    def __next__(self):  
        while True:  
            item = next(self.items)  
  
            # Если ignore_case=True и элемент - строка, приводим к нижнему  
            # регистру для сравнения  
            compare_item = item.lower() if self.ignore_case and isinstance(item, str)  
            else item  
  
            if compare_item not in self.seen:  
                self.seen.add(compare_item)  
                return item  
  
    def __iter__(self):  
        return self  
  
if __name__ == "__main__":  
    # Тесты  
    print("Test 1 - числа:")
```

```

data = [1, 1, 1, 1, 1, 2, 2, 2, 2, 2]
for item in Unique(data):
    print(item, end=' ')
print()

print("\nTest 2 - строки без ignore_case:")
data = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']
for item in Unique(data):
    print(item, end=' ')
print()

print("\nTest 3 - строки с ignore_case=True:")
for item in Unique(data, ignore_case=True):
    print(item, end=' ')
print()

```

вывод на экран

```

Test 1 - один аргумент:
Ковер
Диван для отдыха
Стул

Test 2 - несколько аргументов:
{'title': 'Ковер', 'price': 2000}
{'title': 'Диван для отдыха'}
{'price': 3000}
1
test_2
iv5
test_3
a = 1
b = 2
test_4
1
2

```